

Podsumowanie zmian w projekcie DSL Runtime

1. Stan projektu na początku

Na początku projekt był zrealizowany głównie jako proste MVP do obsługi poleceń biurowych.

System działał mniej więcej tak:

```
polecenie użytkownika  
→ prosty planner  
→ Office DSL  
→ runtime  
→ wykonanie mockowej operacji  
→ raport i audyt
```

Obecny wtedy DSL `office.dsl.v1` opisywał przede wszystkim:

- źródła danych,
- kroki wykonania,
- akcje,
- warunki,
- output,
- potwierdzenia,
- polityki bezpieczeństwa.

Przykładowe obsługiwane operacje:

- pobranie danych,
- wygenerowanie raportu,
- przygotowanie wiadomości,
- mockowe wysłanie wiadomości,
- analiza logów,
- zadanie prostego pytania doprecyzowującego.

Projekt posiadał:

- TypeScript runtime,
- CLI,
- backend API,
- prosty interfejs demonstracyjny,
- mock planner,
- mock Python verifier,
- podstawowy audit log,
- jednostronne potwierdzenia,
- plan hash,
- kilka przykładowych scenariuszy Office DSL.

2. Główne problemy względem pierwotnych ustaleń

Po porównaniu implementacji z uwagami okazało się, że projekt został zinterpretowany zbyt wąsko.

Został zbudowany głównie jako:

Office Workflow DSL

natomiast docelowy system miał służyć przede wszystkim do:

formalizacji intencji człowieka
→ diagnozowania braków
→ wykrywania niejednoznaczności i konfliktów
→ uzgadniania treści pomiędzy Human1 i Human2
→ generowania umów, kontraktów, zleceń, kodu i testów

Najważniejsze brakujące elementy to:

- brak formalnego Intent/Contract DSL,
- brak modelu Human1 i Human2,
- brak dwustronnej akceptacji,
- brak obsługi historii rozmowy dwóch stron,
- brak generowania umów,
- brak pełnego traceability,
- brak statusów typu MISSING, AMBIGUOUS, CONFLICTING,
- brak pełnej walidacji semantycznej NL → DSL,
- brak generowania kodu i testów na podstawie DSL,
- brak pełnego procesu DSL → dokument NL,
- brak runnera testującego wszystkie przykłady po zmianach danych wejściowych.

3. Doprecyzowany cel projektu

Projekt został ponownie opisany zgodnie z docelową wizją.

System ma obsługiwać dwa główne modele.

Human-to-Machine

Pracownik wpisuje polecenie w języku naturalnym.

System:

1. analizuje wypowiedź,
2. generuje formalny DSL,
3. wykrywa braki,
4. zadaje pytania doprecyzowujące,
5. pokazuje użytkownikowi zrozumiałą reprezentację DSL,

6. uzyskuje potwierdzenie,
7. wykonuje zadanie lub generuje kod,
8. zapisuje pełny audyt.

Celem jest uniknięcie sytuacji, w której LLM źle interpretuje kilka nieprecyzyjnych zdań, a użytkownik dowiaduje się o tym dopiero po wykonaniu.

Human-to-Human

System ma formalizować ustalenia pomiędzy dwiema stronami.

Przykładowo:

```
rozmowa Human1 i Human2
→ DSL
→ wykrycie braków i konfliktów
→ pytania do obu stron
→ zatwierdzenie przez Human1
→ zatwierdzenie przez Human2
→ wygenerowanie umowy lub zlecenia
```

Obie strony muszą zaakceptować tę samą wersję DSL.

Każda zmiana DSL powinna unieważniać wcześniejsze zatwierdzenia.

4. Zmieniona rola DSL

DSL nie jest już opisywany wyłącznie jako lista kroków wykonawczych.

Docelowo ma reprezentować między innymi:

- strony,
- role,
- intencje,
- zobowiązania,
- zakres,
- terminy,
- płatności,
- kryteria odbioru,
- wyłączenia,
- założenia,
- konflikty,
- brakujące dane,
- zatwierdzenia,
- źródła informacji,
- reguły renderowania,
- zasady wykonania.

Każde istotne pole ma docelowo posiadać status, np.:

CONFIRMED
MISSING
INCOMPLETE
AMBIGUOUS
CONFLICTING
ASSUMED
REJECTED
NOT_APPLICABLE

Dzięki temu system będzie rozróżniał:

- co zostało podane,
- czego brakuje,
- co jest niejasne,
- co jest sprzeczne,
- co zostało tylko założone przez LLM,
- co zaakceptowały strony.

5. Zmieniona rola runtime

Runtime nie jest już traktowany tylko jako wykonawca akcji.

Został opisany jako centralny orkiestrator procesu.

Docelowo odpowiada za:

- utrzymywanie aktualnego DSL,
- walidację DSL,
- wykrywanie braków i konfliktów,
- zarządzanie stanami procesu,
- kierowanie pytań do Human1 lub Human2,
- aktualizowanie DSL po odpowiedziach,
- obsługę zatwierdzeń,
- obliczanie hasha DSL,
- unieważnianie akceptacji po zmianach,
- renderowanie DSL do dokumentów,
- uruchamianie codegen i testgen,
- wywoływanie Python verifera,
- zapisywanie pełnego audytu.

6. Zmiany w dokumentacji

Została przebudowana dokumentacja projektu.

Dodano nowy dokument architektoniczny

Utworzono:

docs/system-purpose-and-runtime-flow.md

Dokument opisuje:

- problem biznesowy,
- formalizację intencji,
- H2M i H2H,
- rolę DSL,
- rolę LLM,
- rolę runtime,
- rolę Python verifiera,
- NL → DSL,
- DSL → NL,
- Human1/Human2,
- wykrywanie braków,
- wykrywanie konfliktów,
- zatwierdzenia,
- traceability,
- generowanie dokumentów,
- generowanie kodu,
- generowanie testów,
- stan obecny i docelowy.

Dodano również diagramy Mermaid pokazujące pełne przepływy.

Zaktualizowano README

README obecnie jasno oddziela:

- działające Office DSL MVP,
- funkcje mockowe,
- funkcje częściowe,
- docelową architekturę Intent/Contract DSL Runtime.

Dokumentacja nie przedstawia już obecnego MVP jako kompletnego systemu.

Przebudowano TODO

`TODO.md` został podzielony na fazy.

Obejmuje między innymi:

- stabilizację MVP,
- Intent/Contract DSL,
- model braków i konfliktów,
- Human1/Human2,
- NL ↔ DSL,
- renderery dokumentów,
- runner przykładów,
- generowanie testów,

- generowanie kodu,
- semantic verifier,
- CLI Windows/Linux,
- frontend/backend,
- bezpieczeństwo,
- production hardening.

Każde zadanie ma kryterium ukończenia.

Checkbox może zostać oznaczony jako wykonany dopiero po implementacji i testach.

Zaktualizowano HANDOFF

Następny agent otrzymuje teraz jednoznaczną informację:

- jaki jest cel projektu,
- co działa,
- co jest mockiem,
- czego brakuje,
- od którego etapu kontynuować pracę.

7. Nowy system przykładów i walidacji

Został wykonany kolejny etap implementacyjny: runner przykładów.

Dodano pakiet:

```
@office-dsl/example-runner
```

Obecne przykłady posiadają teraz:

- `scenario.json`,
- katalog `in/`,
- katalog `out/`,
- oczekiwane artefakty,
- możliwość uruchomienia pojedynczego scenariusza,
- możliwość uruchomienia wszystkich scenariuszy.

Dostępne komendy:

```
corepack pnpm run example:run -- 01-read-only-report
corepack pnpm run examples:run
corepack pnpm run verify
corepack pnpm run python:test
```

Runner wykonuje:

```
input
→ planner
→ DSL
→ runtime
→ plan
→ Python verifier
→ zapis artefaktów
→ porównanie z expected output
```

Wygenerowane artefakty trafiają do:

```
.office-dsl/generated/examples/<id>/
```

Obecnie wszystkie 6 scenariuszy przechodzi.

8. Poprawki w verifierze

Naprawiono błąd, przez który `email.send` zapisane tylko w polityce mogło zostać potraktowane jako faktycznie wykonana akcja.

Verifier analizuje teraz rzeczywiste akcje, zamiast wyszukiwać samo wystąpienie tekstu w całym DSL.

Dodano test regresyjny dla tego przypadku.

9. Testy i walidacja

Po zmianach przechodzą:

```
TypeScript typecheck
lint
format
17 testów TypeScript
4 testy Python
6/6 przykładów
pełna komenda verify
git diff --check
```

Projekt posiada teraz powtarzalny proces walidacji, który będzie używany przy kolejnych zmianach DSL, runtime'u i verifera.

10. Wersjonowanie i changelog

Pierwszy etap dokumentacyjny zwiększył wersję do:

0.1.1

Po dodaniu pełnego runnera przykładów i komendy `verify` wersja została zwiększona do:

0.2.0

Zaktualizowano:

- `VERSION.md`,
- `CHANGELOG.md`,
- główny `package.json`,
- wersje pakietów workspace,
- `verifier/pyproject.toml`,
- wersję OpenAPI backendu.

11. Co działa obecnie

Gotowe

- Office DSL MVP,
- TypeScript runtime,
- CLI,
- backend API,
- UI demo,
- mock planner,
- mock execution,
- audit,
- pytania `user.ask`,
- jednostronne potwierdzenia,
- hash planu,
- runner przykładów,
- Python verifier w runnerze,
- wspólna komenda `verify`,
- testy regresyjne.

Częściowe

- OpenRouter,
- LiteLLM,
- rendering DSL do tekstu,
- clarification flow,
- audit jako fundament przyszłego traceability.

Mock

- semantyczny planner,
- semantyczny verifier,
- dane,
- wysyłka wiadomości,

- frontend produkcyjny.

12. Czego nadal brakuje

Nie są jeszcze zaimplementowane:

- właściwy Intent/Contract DSL,
- statusy pól,
- source references,
- pełne traceability,
- Human2,
- bilateral approval,
- wspólny hash DSL dla obu stron,
- historia rozmowy dwóch stron,
- umowa zlecenia,
- umowa o pracę,
- formaty prawne,
- DSL → kod JS/Node.js,
- generowanie testów z DSL,
- pełna walidacja semantyczna,
- pełna integracja runtime → Python verifier,
- produkcyjny OpenRouter flow.

13. Najbliższy plan

Najbliższe etapy to:

1. stabilizacja `verify` na Windows i Linux,
2. model statusów pól,
3. source references i traceability,
4. kanoniczna serializacja DSL,
5. hash snapshotu DSL,
6. minimalny model Human1/Human2,
7. bilateral approval,
8. scenariusze rozmów i dokumentów,
9. renderery umów,
10. codegen, testgen i pełny verifier.

14. Podsumowanie

Na początku projekt był głównie działającym Office Workflow DSL MVP.

Po analizie został dopasowany do właściwej wizji:

```
formalizacja intencji
→ diagnozowanie braków
→ uzgadnianie pomiędzy Human1 i Human2
→ zatwierdzony DSL
```

- umowa, zlecenie, kod lub testy
- walidacja i audyt

Na obecnym etapie została poprawiona architektura, dokumentacja, roadmapa i proces testowania.

Nie oznacza to jeszcze, że cały docelowy system jest gotowy.

Obecny stan to:

```
stabilne Office DSL MVP
+
prawidłowo opisana docelowa architektura
+
powtarzalny runner i walidacja
+
gotowa roadmapa dalszej implementacji
```